

ACARA PRAKTIKUM 9

ALGORITMA GENETIKA 1

TUJUAN

- Mengetahui dasar-dasar Algoritma Genetika
- Mengetahui dasar-dasar Cara Kerja Algoritma Genetika
- Mengetahui penerapan Algoritma Genetika dalam kasus nyata.
- Terampil menerapkan Algoritma Genetika dalam kasus nyata.

DASAR TEORI

1. Algoritma Genetika (Algen) dan Sejarahnya

Algoritma Genetika (Algen) merupakan salah satu metode pencarian dan optimasi berbasis populasi yang terinspirasi oleh proses evolusi biologis, seperti seleksi alam dan pewarisan genetik. Konsep ini pertama kali dikembangkan oleh John Holland pada tahun 1960-an di University of Michigan. Holland mengadaptasi prinsip-prinsip dari teori Darwin tentang seleksi alam dan menerapkannya dalam konteks komputasi untuk menyelesaikan masalah optimasi yang kompleks. Penelitian ini semakin berkembang, terutama setelah bukunya *Adaptation in Natural and Artificial Systems* terbit pada tahun 1975.

2. Pengertian dan Latar Belakang Algoritma Genetika

Secara umum, Algoritma Genetika adalah metode pencarian solusi optimal yang terinspirasi dari proses evolusi makhluk hidup. Algen bekerja dengan menggunakan populasi solusi sementara yang terus berevolusi dari generasi ke generasi melalui operasi genetik seperti seleksi, crossover, dan mutasi. Alasan utama pengembangan algoritma ini adalah karena beberapa masalah optimasi yang sangat kompleks tidak dapat diselesaikan dengan metode deterministik biasa. Oleh karena itu, Algoritma Genetika dikembangkan untuk menangani masalah optimasi di mana solusi yang tepat sulit ditemukan dengan pencarian eksak.

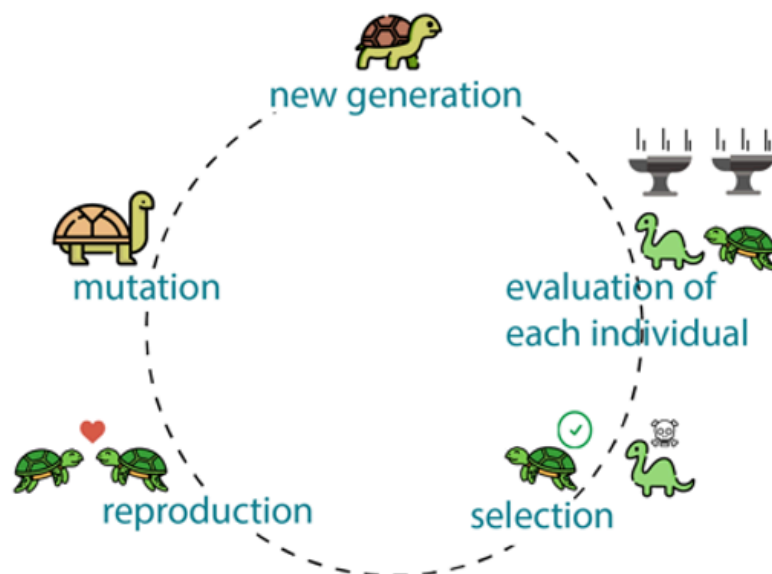
3. Cara Kerja Algoritma Genetika

Algoritma Genetika dimulai dengan membangkitkan populasi awal yang berisi kumpulan individu, di mana setiap individu mewakili solusi potensial. Setiap individu

dinilai berdasarkan fitness function, yang mengukur seberapa baik solusi tersebut dalam memecahkan masalah yang dihadapi. Selanjutnya, algoritma melakukan seleksi, di mana individu dengan fitness terbaik dipilih untuk menghasilkan keturunan melalui operasi crossover dan mutasi. Proses ini terus diulang untuk beberapa generasi, dengan harapan solusi yang lebih baik akan muncul seiring berjalannya waktu.

Langkah-langkah utama dalam Algoritma Genetika meliputi:

1. Inisialisasi :Membuat populasi awal secara acak.
2. Evaluasi :Menghitung fitness setiap individu.
3. Seleksi :Memilih individu terbaik untuk menghasilkan keturunan.
4. Crossover :Menggabungkan dua individu untuk menghasilkan keturunan baru.
5. Mutasi :Mengubah beberapa bagian dari individu untuk menjaga keragaman populasi.
6. Generasi Baru :Menggantikan populasi lama dengan generasi baru.



4. Contoh Studi Kasus Algoritma Genetika

Algoritma Genetika telah diterapkan dalam berbagai studi kasus, terutama pada masalah optimasi seperti Knapsack Problem dan Traveling Salesman Problem (TSP), kasus Penjadwalan, dan lain sebagainya.

1. Knapsack Problem: Dalam masalah ini, contohnya seseorang harus memilih barang-barang yang akan dimasukkan ke dalam tas dengan kapasitas terbatas, di mana setiap barang memiliki bobot dan nilai. Tujuan Algoritma Genetika dalam konteks ini adalah menemukan kombinasi barang yang memberikan nilai maksimum tanpa melebihi kapasitas tas.
2. Traveling Salesman Problem (TSP): TSP adalah masalah optimasi rute di mana seorang penjual harus mengunjungi beberapa kota dengan jarak tertentu dan kembali ke kota asal, dengan tujuan meminimalkan total jarak perjalanan. Algoritma Genetika digunakan untuk mencari urutan kota yang optimal.
3. Kasus Penjadwalan

Salah satu aplikasi penting Algoritma Genetika adalah dalam menyelesaikan masalah penjadwalan, yang sering kali kompleks karena melibatkan banyak variabel dan batasan. Penjadwalan adalah proses mengatur sumber daya, seperti waktu, tenaga kerja, atau mesin, untuk melakukan serangkaian tugas dalam waktu yang optimal. Contoh yang paling umum dari masalah ini adalah Penjadwalan Karyawan, Penjadwalan Proyek, dan Penjadwalan Produksi di Pabrik.

 - a. Penjadwalan Karyawan: Dalam masalah ini, sebuah perusahaan harus menentukan jadwal kerja bagi karyawannya, dengan memperhatikan berbagai batasan, seperti jumlah jam kerja maksimal per minggu, hari libur, atau keahlian yang dibutuhkan pada jam tertentu. Tujuan Algoritma Genetika adalah menemukan jadwal yang memenuhi semua batasan ini sekaligus memaksimalkan efisiensi tenaga kerja. Dengan melakukan seleksi, crossover, dan mutasi pada kumpulan solusi (jadwal), Algoritma Genetika dapat menghasilkan jadwal yang optimal atau mendekati optimal yang mengurangi waktu penyusunan jadwal secara manual.
 - b. Penjadwalan Proyek: Dalam konteks manajemen proyek, penjadwalan melibatkan pengalokasian sumber daya (misalnya, pekerja dan peralatan) untuk menyelesaikan serangkaian tugas proyek dalam batasan waktu dan anggaran tertentu. Masalah ini sering dikenal dengan Project Scheduling Problem atau Job Shop Scheduling Problem. Algoritma Genetika diterapkan untuk mengoptimalkan

urutan pengerjaan tugas, sehingga waktu penyelesaian proyek dapat diminimalkan dan penggunaan sumber daya dapat dioptimalkan.

- c. Penjadwalan Produksi di Pabrik: Masalah penjadwalan produksi di pabrik melibatkan penentuan urutan tugas produksi untuk mesin-mesin yang ada dengan tujuan mengoptimalkan waktu proses dan mengurangi waktu idle mesin. Algoritma Genetika dapat digunakan untuk memecahkan masalah ini dengan mencari urutan tugas yang meminimalkan waktu total penyelesaian (makespan), serta memaksimalkan produktivitas pabrik.

5. Hubungan Algoritma Genetika dengan Knapsack

Contoh: Knapsack problem, Teknik memasukkan item ke dalam suatu wadah dengan nilai profit paling maksimal.



Pilih barang mana saja ?

Dalam masalah Knapsack, Algoritma Genetika sangat efektif digunakan karena mampu menemukan solusi optimal atau mendekati optimal untuk masalah dengan jumlah variabel yang sangat besar. Proses crossover dan mutasi memungkinkan eksplorasi ruang solusi yang lebih luas, sementara seleksi memastikan bahwa solusi terbaik tetap bertahan dalam evolusi generasi berikutnya. Algoritma Genetika dapat digunakan dalam skenario di mana terdapat banyak pilihan barang dan batasan kapasitas, dengan tujuan untuk memaksimalkan profit atau utilitas dari barang yang dipilih.

Dengan Algen, tujuan utamanya adalah untuk menemukan kombinasi solusi terbaik dengan waktu komputasi yang lebih efisien dibandingkan metode pencarian lengkap. Ini membuat Algen sangat berguna dalam konteks masalah optimasi yang sangat besar dan kompleks.

ALAT DAN BAHAN

1. Code Editor (Visual Studio Code / Cursor Code Editor / Notepad / Sublime Text / dsb)
2. Environment Python yang telah siap digunakan
3. Library matplotlib

Pastikan Anda telah menginstal **matplotlib** untuk plotting grafik dengan menjalankan **`pip install matplotlib`** jika belum terinstal.

4. Library tkinter

Library Tkinter merupakan library python untuk membuat aplikasi dalam bentuk GUI.

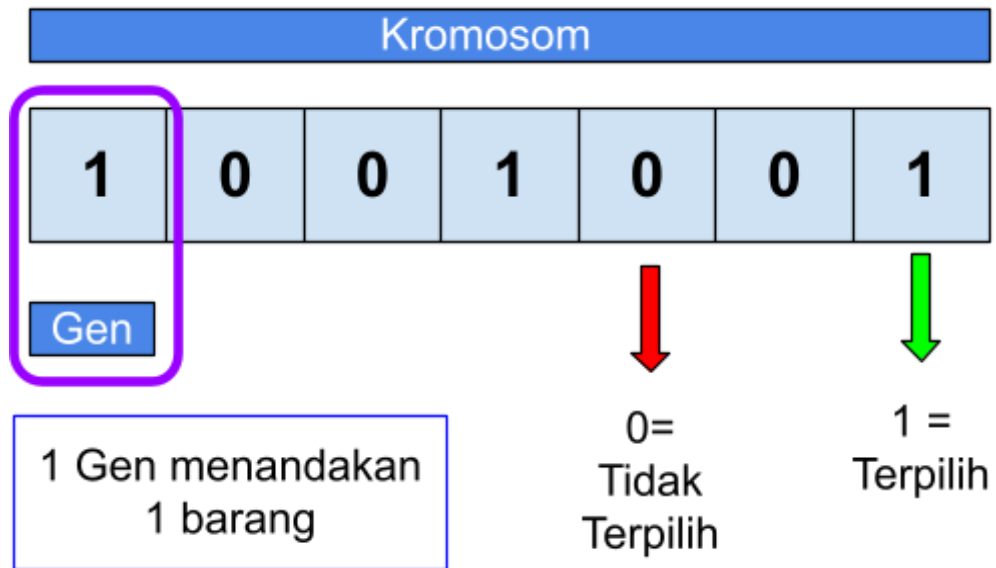
PERCOBAAN

1. Proses Inisiasi Populasi

Proses inisiasi populasi adalah langkah pertama dalam Algoritma Genetika (AG) di mana kita membangkitkan sejumlah individu (kromosom) secara acak untuk membentuk populasi awal. Setiap individu merepresentasikan solusi potensial terhadap masalah yang ingin kita selesaikan, dalam hal ini adalah Knapsack Problem.

Dalam Knapsack Problem, kita memiliki sejumlah barang dengan nilai (harga) dan bobot tertentu. Tujuannya adalah memilih kombinasi barang yang dapat dimasukkan ke dalam tas dengan kapasitas terbatas untuk memaksimalkan total nilai barang tanpa melebihi kapasitas tas.

Setiap kromosom direpresentasikan sebagai array biner, di mana setiap gen menunjukkan apakah barang tersebut dipilih (1) atau tidak (0). Jika kita memiliki N barang, maka setiap kromosom akan memiliki N gen.



Contoh Cara Kerja:

Misalkan kita memiliki 5 barang. Kita ingin membangkitkan populasi awal dengan 10 individu. Setiap individu akan memiliki 5 gen yang dibangkitkan secara acak.

Contoh kromosom:

1. **Individu 1:** [1, 0, 1, 0, 1] (Barang ke-1, ke-3, dan ke-5 dipilih)
2. **Individu 2:** [0, 1, 0, 1, 0] (Barang ke-2 dan ke-4 dipilih)

Buat Sebuah Folder dengan nama Algoritma Genetika, lalu tambahkan file dengan Nama File `InisiasiPopulasi.py`, kemudian ketikkan kode berikut

```
import random

# Fungsi untuk inisialisasi populasi
def inisialisasi_populasi(jumlah_populasi, jumlah_gen):
    populasi = []
    for i in range(jumlah_populasi):
        # Membuat kromosom dengan gen biner secara acak
        kromosom = [random.randint(0, 1) for _ in
range(jumlah_gen)]
        populasi.append(kromosom)
    return populasi

# Contoh penggunaan
jumlah_populasi = 10 # Jumlah individu dalam populasi
jumlah_gen = 5       # Jumlah barang (gen) dalam kromosom
```

```
populasi_awal = inisialisasi_populasi(jumlah_populasi,
jumlah_gen)

# Menampilkan populasi awal
print("Populasi Awal:")
for idx, individu in enumerate(populasi_awal):
    print(f"Individu {idx+1}: {individu}")
```

Penjelasan Kode Python secara Detail:

- **Import Modul `random`:** Modul ini digunakan untuk membangkitkan angka acak dalam proses inisialisasi populasi.
- **Fungsi `inisialisasi_populasi`:**
 - **Parameter:**
 - `jumlah_populasi`: Jumlah individu yang ingin dibangkitkan dalam populasi awal.
 - `jumlah_gen`: Jumlah gen dalam setiap kromosom, sesuai dengan jumlah barang yang ada.
 - **Proses:**
 - Membuat list kosong `populasi` untuk menyimpan individu-individu.
 - Menggunakan loop `for` untuk membangkitkan `jumlah_populasi` individu.
 - Dalam setiap iterasi, membangkitkan kromosom dengan gen biner secara acak menggunakan list comprehension
`[random.randint(0, 1) for _ in range(jumlah_gen)]`.
 - Menambahkan kromosom tersebut ke dalam populasi.
 - **Return:**
 - Mengembalikan list `populasi` yang berisi individu-individu dengan kromosom acak.
- **Contoh Penggunaan:**
 - Menentukan `jumlah_populasi` dan `jumlah_gen` sesuai kebutuhan.

- Memanggil fungsi `inisialisasi_populasi` untuk membangkitkan populasi awal.
- Menampilkan populasi awal dengan mencetak setiap individu beserta kromosomnya.

Silahkan Running dan hasilnya akan memunculkan seperti berikut.

```
Populasi Awal:
Individu 1: [1, 1, 0, 0, 1]
Individu 2: [1, 0, 1, 0, 0]
Individu 3: [0, 1, 1, 0, 1]
Individu 4: [1, 0, 1, 0, 0]
Individu 5: [1, 1, 0, 1, 1]
Individu 6: [1, 1, 1, 0, 0]
Individu 7: [1, 0, 1, 1, 0]
Individu 8: [0, 1, 1, 1, 1]
Individu 9: [1, 0, 1, 0, 0]
Individu 10: [0, 1, 0, 0, 1]
```

2. Proses Evaluasi Fitness

Evaluasi fitness adalah proses menghitung nilai fitness untuk setiap individu dalam populasi. Nilai fitness menggambarkan seberapa baik solusi (kromosom) tersebut dalam menyelesaikan masalah yang diberikan.

Dalam Knapsack Problem, nilai fitness dapat didefinisikan sebagai total nilai (harga) barang yang dipilih dalam kromosom, dengan syarat total bobot barang tidak melebihi kapasitas tas. Jika total bobot melebihi kapasitas, maka individu tersebut dianggap tidak valid dan diberi penalti (fitness = 0).

Contoh Cara Kerja:

Misalkan kita memiliki data barang sebagai berikut:

Barang	Harga	Bobot
Barang1	60	10
Barang2	100	20

Barang3	120	30
Barang4	90	25
Barang5	70	15

Kapasitas tas adalah 50.

Individu: [1, 0, 1, 0, 1]

- Barang yang dipilih: Barang1, Barang3, Barang5
- Total harga: $60 + 120 + 70 = 250$
- Total bobot: $10 + 30 + 15 = 55$
- Karena total bobot (55) melebihi kapasitas tas (50), maka fitness = 0 (penalti).

Individu: [0, 1, 0, 1, 0]

- Barang yang dipilih: Barang2, Barang4
- Total harga: $100 + 90 = 190$
- Total bobot: $20 + 25 = 45$
- Total bobot tidak melebihi kapasitas, sehingga fitness = 190.

Pada Sebuah Folder dengan nama Algoritma Genetika , tambahkan file dengan Nama File `EvaluasiFitness.py`, kemudian ketikkan kode berikut

```
# Data barang (nama, harga, bobot)
barang = [
    ("Barang1", 60, 10),
    ("Barang2", 100, 20),
    ("Barang3", 120, 30),
    ("Barang4", 90, 25),
    ("Barang5", 70, 15)
]

kapasitas_tas = 50 # Kapasitas maksimum tas

# Fungsi untuk menghitung nilai fitness
def hitung_fitness(kromosom, barang, kapasitas_tas):
    total_harga = 0
    total_bobot = 0
```

```

for i in range(len(kromosom)):
    if kromosom[i] == 1:
        total_harga += barang[i][1]
        total_bobot += barang[i][2]
    if total_bobot > kapasitas_tas:
        return 0 # Penalti jika melebihi kapasitas
    else:
        return total_harga

# Definisi contoh populasi awal
populasi_awal = [
    [1, 0, 1, 0, 1], # Contoh kromosom individu
    [0, 1, 0, 1, 0],
    [1, 1, 0, 0, 1],
    # Tambahkan lebih banyak individu sesuai kebutuhan
]

# Contoh penggunaan
fitness_populasi = [hitung_fitness(individu, barang,
    kapasitas_tas) for individu in populasi_awal]

# Menampilkan nilai fitness
print("\nNilai Fitness:")
for idx, fitness in enumerate(fitness_populasi):
    print(f"Individu {idx+1}: Fitness = {fitness}")

```

Penjelasan Kode Python secara Detail:

- **Data Barang:**
 - List **barang** berisi tuple yang merepresentasikan setiap barang dengan format **(nama, harga, bobot)**.
 - Contoh: **("Barang1", 60, 10)** berarti Barang1 memiliki harga 60 dan bobot 10.
- **Kapasitas Tas:**
 - Variabel **kapasitas_tas** menentukan kapasitas maksimum tas, yaitu 50.
- **Fungsi **hitung_fitness**:**
 - **Parameter:**
 - **kromosom**: List gen yang merepresentasikan individu.
 - **Proses:**

- Menginisialisasi `total_harga` dan `total_bobot` dengan nilai 0.
- Menggunakan loop `for` untuk memeriksa setiap gen dalam kromosom.
- Jika gen bernilai 1 (barang dipilih), maka:
 - Menambahkan harga barang ke `total_harga`.
 - Menambahkan bobot barang ke `total_bobot`.
- Setelah semua gen diperiksa, memeriksa apakah `total_bobot` melebihi `kapasitas_tas`:
 - Jika ya, mengembalikan nilai fitness 0 sebagai penalti.
 - Jika tidak, mengembalikan `total_harga` sebagai nilai fitness.
- **Return:**
 - Nilai fitness (0 atau total harga barang yang dipilih).
- **Contoh Penggunaan:**
 - Menghitung nilai fitness untuk setiap individu dalam `populasi_awal` menggunakan list comprehension.
 - Menampilkan nilai fitness setiap individu dengan mencetaknya.

Hasil dari Kode tersebut akan menyerupai sebagai berikut.

```

Nilai Fitness:
Individu 1: Fitness = 0
Individu 2: Fitness = 190
Individu 3: Fitness = 230
  
```

3. Proses Seleksi

Proses seleksi bertujuan untuk memilih individu-individu yang akan menjadi orang tua (parent) untuk menghasilkan generasi berikutnya melalui proses crossover dan mutasi. Seleksi dilakukan berdasarkan nilai fitness, di mana individu dengan fitness lebih tinggi memiliki peluang lebih besar untuk dipilih.

Proses seleksi bertujuan untuk memilih individu-individu yang akan menjadi orang tua (parent) untuk menghasilkan generasi berikutnya melalui proses crossover

dan mutasi. Seleksi dilakukan berdasarkan nilai fitness, di mana individu dengan fitness lebih tinggi memiliki peluang lebih besar untuk dipilih.

Metode seleksi yang umum digunakan:

- **Roulette Wheel Selection:** Individu dipilih secara probabilistik berdasarkan proporsi nilai fitness terhadap total fitness populasi.
- **Tournament Selection:** Beberapa individu dipilih secara acak, kemudian individu dengan fitness tertinggi di antara mereka dipilih sebagai parent.

Contoh Cara Kerja:

A. Roulette Wheel Selection:

Roulette Wheel Selection adalah metode seleksi dalam Algoritma Genetika yang memilih individu sebagai parent berdasarkan probabilitas yang proporsional terhadap nilai fitness mereka. Individu dengan nilai fitness lebih tinggi memiliki peluang lebih besar untuk dipilih, tetapi individu dengan nilai fitness lebih rendah masih memiliki kesempatan.

Konsep Dasar:

- Bayangkan sebuah roda roulette (roda keberuntungan) yang dibagi menjadi beberapa sektor, di mana setiap sektor mewakili satu individu dalam populasi.
- Luas sektor untuk setiap individu proporsional terhadap nilai fitness individu tersebut.
- Semakin tinggi nilai fitness, semakin besar luas sektornya, dan semakin besar peluangnya untuk dipilih.
- Proses seleksi dilakukan dengan memutar roda dan melihat pada sektor mana roda berhenti. Individu yang terpilih adalah individu yang sektornya terkena penunjuk.

Langkah-langkah:

1. **Menghitung Total Fitness Populasi:**
 - Menjumlahkan semua nilai fitness individu dalam populasi.
2. **Menghitung Probabilitas Seleksi untuk Setiap Individu:**

- Probabilitas seleksi = (Fitness individu) / (Total fitness populasi)
- 3. **Menghitung Probabilitas Kumulatif:**
 - Menjumlahkan probabilitas seleksi secara kumulatif untuk setiap individu.
- 4. **Menghasilkan Bilangan Acak antara 0 dan 1:**
 - Bilangan acak ini akan digunakan untuk menentukan individu yang terpilih.
- 5. **Menentukan Individu yang Terpilih:**
 - Jika bilangan acak kurang dari atau sama dengan probabilitas kumulatif individu tersebut, maka individu tersebut dipilih.

Contoh:

Misalkan kita memiliki populasi dengan 4 individu dan nilai fitness sebagai berikut:

- **Individu 1:** Fitness = 1
- **Individu 2:** Fitness = 2
- **Individu 3:** Fitness = 3
- **Individu 4:** Fitness = 4

Langkah 1: Menghitung Total Fitness Populasi

Total fitness = $1 + 2 + 3 + 4 = 10$

Langkah 2: Menghitung Probabilitas Seleksi untuk Setiap Individu

- Probabilitas Individu 1 = $1 / 10 = 0.1$
- Probabilitas Individu 2 = $2 / 10 = 0.2$
- Probabilitas Individu 3 = $3 / 10 = 0.3$
- Probabilitas Individu 4 = $4 / 10 = 0.4$

Langkah 3: Menghitung Probabilitas Kumulatif

- Probabilitas Kumulatif Individu 1 = 0.1
- Probabilitas Kumulatif Individu 2 = $0.1 + 0.2 = 0.3$
- Probabilitas Kumulatif Individu 3 = $0.3 + 0.3 = 0.6$
- Probabilitas Kumulatif Individu 4 = $0.6 + 0.4 = 1.0$

Langkah 4: Menghasilkan Bilangan Acak

Misalkan kita menghasilkan bilangan acak $r = 0.45$.

Langkah 5: Menentukan Individu yang Terpilih

- **Individu 1:**
 - Probabilitas Kumulatif = 0.1
 - Karena $r (0.45) > 0.1$, Individu 1 tidak terpilih.
- **Individu 2:**
 - Probabilitas Kumulatif = 0.3
 - Karena $r (0.45) > 0.3$, Individu 2 tidak terpilih.
- **Individu 3:**
 - Probabilitas Kumulatif = 0.6
 - Karena $r (0.45) \leq 0.6$, **Individu 3 terpilih.**
- Tidak perlu memeriksa Individu 4 karena sudah menemukan individu yang memenuhi kondisi.

Hasil:

- **Individu 3** dipilih sebagai parent.

Mengapa Individu dengan Fitness Lebih Tinggi Memiliki Peluang Lebih Besar?

- Individu 4 memiliki probabilitas seleksi tertinggi (0.4), sehingga memiliki peluang 40% untuk dipilih.
- Individu 1 memiliki probabilitas seleksi terendah (0.1), dengan peluang 10% untuk dipilih.
- Metode ini memastikan individu dengan nilai fitness lebih tinggi memiliki peluang lebih besar, tetapi tidak menghilangkan kesempatan bagi individu dengan fitness lebih rendah.

B. Tournament Selection:

Dalam Tournament Selection, proses seleksi dilakukan dengan cara memilih beberapa individu secara acak dari populasi (disebut peserta turnamen), kemudian individu dengan nilai fitness tertinggi di antara mereka dipilih sebagai parent.

Langkah-langkah:

1. Menentukan Ukuran Turnamen (**k**):

- Misalkan kita memilih **k = 3**, artinya setiap turnamen akan melibatkan 3 peserta yang dipilih secara acak dari populasi.

2. Memilih Peserta Turnamen:

- Dari populasi, pilih 3 individu secara acak tanpa memperhatikan nilai fitness mereka.

3. Menentukan Pemenang Turnamen:

- Di antara peserta yang terpilih, bandingkan nilai fitness mereka.
- Individu dengan nilai fitness tertinggi di antara peserta dipilih sebagai parent.

Contoh:

Misalkan kita memiliki populasi dengan 6 individu dan nilai fitness mereka sebagai berikut:

- **Individu 1:** Fitness = 30
- **Individu 2:** Fitness = 50
- **Individu 3:** Fitness = 70
- **Individu 4:** Fitness = 60
- **Individu 5:** Fitness = 80
- **Individu 6:** Fitness = 40

Langkah 1: Menentukan Ukuran Turnamen

- Kita memilih ukuran turnamen **k = 3**.

Langkah 2: Memilih Peserta Turnamen

- Secara acak, misalkan kita memilih **Individu 2**, **Individu 4**, dan **Individu 6** sebagai peserta turnamen.

Langkah 3: Menentukan Pemenang Turnamen

- Nilai fitness peserta:
 - **Individu 2:** Fitness = 50
 - **Individu 4:** Fitness = 60
 - **Individu 6:** Fitness = 40

- Pemenang turnamen adalah **Individu 4** dengan fitness tertinggi (60).

Hasil:

- **Individu 4** dipilih sebagai parent pertama.

Proses diulangi untuk memilih parent kedua dengan langkah yang sama.

Keuntungan Tournament Selection:

- **Selektif Terhadap Fitness Tinggi:** Individu dengan fitness tinggi memiliki peluang lebih besar untuk dipilih, tetapi individu dengan fitness lebih rendah masih memiliki kesempatan.
- **Kontrol Tekanan Seleksi:** Ukuran turnamen **k** mempengaruhi tekanan seleksi. Semakin besar **k**, semakin tinggi kemungkinan individu dengan fitness tinggi dipilih.

Pada Sebuah Folder dengan nama Algoritma Genetika , tambahkan file dengan Nama File `selection.py`, kemudian ketikkan kode berikut

```
import random

# Fungsi untuk Roulette Wheel Selection
def roulette_wheel_selection(populasi, fitness_populasi):
    total_fitness = sum(fitness_populasi)
    if total_fitness == 0:
        idx = random.randrange(len(populasi))
        return populasi[idx], idx # Mengembalikan individu
    dan indeks nya
    probabilitas = [fitness / total_fitness for fitness in
fitness_populasi]
    kumulatif_prob = []
    kumulatif = 0
    for p in probabilitas:
        kumulatif += p
        kumulatif_prob.append(kumulatif)
    r = random.random()
    for i, kum_prob in enumerate(kumulatif_prob):
        if r <= kum_prob:
            return populasi[i], i # Mengembalikan individu
    dan indeks nya
    return populasi[-1], len(populasi)-1 # Jika tidak ada
yang memenuhi, kembalikan individu terakhir
```



```

# Fungsi untuk Tournament Selection
def tournament_selection(populasi, fitness_populasi, k=3):
    if len(populasi) < k:
        k = len(populasi)
    peserta_indices = random.sample(range(len(populasi)), k)
    peserta = [(populasi[i], fitness_populasi[i], i) for i in
peserta_indices]
    peserta.sort(key=lambda x: x[1], reverse=True)
    return peserta[0][0], peserta[0][2] # Mengembalikan
individu dan indeksinya

# Definisikan populasi awal dan fitness_populasi
populasi_awal = ['individu1', 'individu2', 'individu3',
'individu4']
fitness_populasi = [10, 20, 30, 40]

# Membuat salinan populasi dan fitness untuk dimodifikasi
available_populasi = populasi_awal.copy()
available_fitness = fitness_populasi.copy()

# Contoh penggunaan
# Memilih Parent 1 menggunakan Roulette Wheel Selection
parent1, idx1 = roulette_wheel_selection(available_populasi,
available_fitness)
# Menghapus parent1 dari daftar available_populasi dan
available_fitness
del available_populasi[idx1]
del available_fitness[idx1]

# Memilih Parent 2 menggunakan Tournament Selection
parent2, idx2 = tournament_selection(available_populasi,
available_fitness)
# Menghapus parent2 dari daftar available_populasi dan
available_fitness
del available_populasi[idx2]
del available_fitness[idx2]

print("\nParent Terpilih:")
print(f"Parent 1: {parent1}")
print(f"Parent 2: {parent2}")

```

Penjelasan Kode Python secara Detail:

- Fungsi **roulette_wheel_selection**:
 - Parameter:

- `populasi`: List individu dalam populasi.
 - `fitness_populasi`: List nilai fitness masing-masing individu.
- **Proses:**
 - Menghitung `total_fitness` dengan menjumlahkan semua nilai fitness dalam populasi.
 - Jika `total_fitness` adalah 0 (semua fitness 0), maka memilih individu secara acak.
 - Menghitung probabilitas pemilihan setiap individu dengan membagi nilai fitness individu dengan `total_fitness`.
 - Membuat list `kumulatif_prob` yang berisi probabilitas kumulatif.
 - Menghasilkan bilangan acak `r` antara 0 dan 1.
 - Menentukan individu yang terpilih dengan membandingkan `r` dengan `kumulatif_prob`.
- **Return:**
 - Individu yang terpilih sebagai parent.
- **Fungsi `tournament_selection`:**
 - **Parameter:**
 - `populasi`: List individu dalam populasi.
 - `fitness_populasi`: List nilai fitness masing-masing individu.
 - `k`: Jumlah individu yang dipilih untuk turnamen (default 3).
 - **Proses:**
 - Memilih `k` individu secara acak dari populasi menggunakan `random.sample`.
 - Mengurutkan peserta turnamen berdasarkan nilai fitness secara menurun.
 - Memilih individu dengan fitness tertinggi sebagai pemenang turnamen.
 - **Return:**
 - Individu yang terpilih sebagai parent.
- **Contoh Penggunaan:**
 - Memilih `parent1` menggunakan metode Roulette Wheel Selection.
 - Memilih `parent2` menggunakan metode Tournament Selection.

- Menampilkan kromosom parent yang terpilih.

Hasil dari kode tersebut akan menyerupai sebagai berikut.

```
Parent Terpilih:
Parent 1: individu4
Parent 2: individu3
```

atau, dan lain sebagainya.

```
Parent Terpilih:
Parent 1: individu1
Parent 2: individu4
```

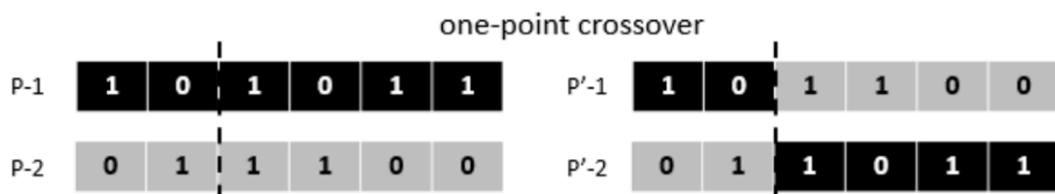
4. Proses Crossover

Crossover (juga dikenal sebagai rekombinasi atau kawin silang) adalah tahap penting dalam algoritma genetika. Berikut adalah beberapa poin terkait proses crossover:

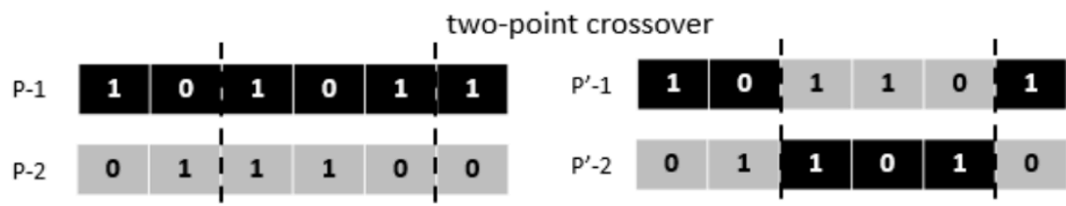
- Setiap kromosom dari orang tua dapat menghasilkan keturunan (offspring) melalui mekanisme crossover.
- Tujuan utama dari crossover adalah menghasilkan kumpulan keturunan (populasi) yang lebih baik daripada generasi sebelumnya.
- Setiap orang tua dipilih melalui operator seleksi, seperti Roulette Wheel Selection (RWS), Tournament Selection (TS), atau Rank Selection (RS).

Beberapa metode crossover:

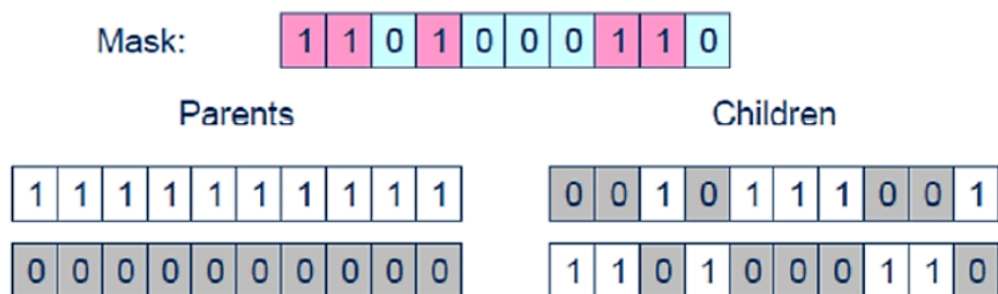
- **One-Point Crossover:** Memotong kromosom di satu titik dan menukar bagian setelah titik tersebut antara kedua parent.



- **Two-Point Crossover:** Memotong kromosom di dua titik dan menukar segmen di antara kedua titik tersebut.



- **Uniform Crossover:** Setiap gen pada anak dipilih secara acak dari salah satu parent.



- Buat dulu pola masking pertukaran gen-nya
- Aturan: gen masking 1 -> tukar, gen masking 0 -> tidak ditukar
- Jika gen Parent 1 tersebut menemui angka 0 pada indeks mask yang sama dengan gen tersebut maka Child 1 akan memasukkan nilai indeks Parent 1, begitupun dengan Parent 2 dan Child 2.
- Jika gen Parent 1 tersebut menemui angka 1 pada indeks mask yang sama dengan gen tersebut maka Child 1 akan memasukkan nilai indeks Parent 2, dan Child 2 akan memasukkan nilai indeks Parent 1.
- Note: pola masking dapat di-generate ulang setiap generasi

Pada Sebuah Folder dengan nama Algoritma Genetika , tambahkan file dengan Nama File `crossover.py`, kemudian ketikkan kode berikut

```
import random

# One-Point Crossover
def one_point_crossover(parent1, parent2):
    titik_potong = random.randint(1, len(parent1)-1)
    anak1 = parent1[:titik_potong] + parent2[titik_potong:]
```

```

    anak2 = parent2[:titik_potong] + parent1[titik_potong:]
    return anak1, anak2

# Two-Point Crossover
def two_point_crossover(parent1, parent2):
    titik1 = random.randint(1, len(parent1)-2)
    titik2 = random.randint(titik1+1, len(parent1)-1)
    anak1 = parent1[:titik1] + parent2[titik1:titik2] +
parent1[titik2:]
    anak2 = parent2[:titik1] + parent1[titik1:titik2] +
parent2[titik2:]
    return anak1, anak2

# Uniform Crossover
def uniform_crossover(parent1, parent2):
    # Membuat mask acak
    mask = [random.randint(0, 1) for _ in
range(len(parent1))]
    anak1 = []
    anak2 = []
    for i in range(len(parent1)):
        if mask[i] == 0:
            # Jika mask bernilai 0, ambil gen dari parent1
            # untuk anak1, dan parent2 untuk anak2
            anak1.append(parent1[i])
            anak2.append(parent2[i])
        else:
            # Jika mask bernilai 1, ambil gen dari parent2
            # untuk anak1, dan parent1 untuk anak2
            anak1.append(parent2[i])
            anak2.append(parent1[i])
    return anak1, anak2

# Contoh penggunaan
parent1 = [1, 0, 1, 1, 0] # Contoh parent1
parent2 = [0, 1, 0, 0, 1] # Contoh parent2

anak1, anak2 = one_point_crossover(parent1, parent2)

print("\nAnak Hasil Crossover:")
print(f"Anak 1: {anak1}")
print(f"Anak 2: {anak2}")

```

Penjelasan Kode Python secara Detail:

- **One-Point Crossover:**

- **Parameter:**
 - `parent1`, `parent2`: Kromosom parent yang akan dicrossover.
- **Proses:**
 - Memilih `titik_potong` secara acak antara 1 dan `len(parent1) - 1`.
 - Membuat `anak1` dengan menggabungkan gen dari `parent1` sebelum `titik_potong` dan gen dari `parent2` setelah `titik_potong`.
 - Membuat `anak2` dengan cara sebaliknya.
- **Return:**
 - Dua kromosom anak hasil crossover.
- **Two-Point Crossover:**
 - Memilih dua titik potong `titik1` dan `titik2` secara acak.
 - Menukar segmen gen antara kedua titik potong.
- **Uniform Crossover:**
 - Dalam contoh ini, kita menetapkan mask secara acak
 - Untuk setiap gen, ditentukan berdasarkan indeks yang sama dengan mask apakah gen tersebut akan diambil dari `parent1` atau `parent2`.
 - Jika gen Parent 1 tersebut menemui angka 0 pada indeks mask yang sama dengan gen tersebut maka Child 1 akan memasukkan nilai indeks Parent 1, begitupun dengan Parent 2 dan Child 2.
 - Jika gen Parent 1 tersebut menemui angka 1 pada indeks mask yang sama dengan gen tersebut maka Child 1 akan memasukkan nilai indeks Parent 2, dan Child 2 akan memasukkan nilai indeks Parent 1.
- **Contoh Penggunaan:**
 - Menggunakan `one_point_crossover` untuk menghasilkan `anak1` dan `anak2`.
 - Menampilkan kromosom anak yang dihasilkan.

Hasil dari kode tersebut akan menyerupai sebagai berikut dan bisa juga acak.

Anak Hasil Crossover:

Anak 1: [1, 0, 1, 0, 1] Anak 2: [0, 1, 0, 1, 0]
--

5. Proses Mutasi

Mutasi adalah proses melakukan perubahan kecil pada kromosom individu untuk mempertahankan keragaman genetik dalam populasi dan mencegah konvergensi prematur. Mutasi dilakukan dengan mengubah satu atau beberapa gen dalam kromosom.

Metode mutasi yang umum digunakan:

- Swap Mutation: Menukar posisi dua gen dalam kromosom.
- Inversion Mutation: Membalik urutan gen dalam segmen tertentu.
- Uniform Mutation: Mengubah nilai gen dengan probabilitas tertentu.

Contoh Cara Kerja:

- **Swap Mutation:**
 - **Kromosom sebelum mutasi:** [1, 0, 1, 1, 0]
 - Menukar gen ke-2 dan ke-4.
 - **Kromosom setelah mutasi:** [1, 1, 1, 0, 0]

Pada Sebuah Folder dengan nama Algoritma Genetika , tambahkan file dengan Nama File `mutation.py`, kemudian ketikkan kode berikut

```
import random

# Swap Mutation
def swap_mutation(kromosom):
    # Pastikan kromosom adalah list
    kromosom = list(kromosom) # Konversi ke list jika perlu

    # Pilih dua posisi untuk swap
    posisi1, posisi2 = random.sample(range(len(kromosom)), 2)

    # Melakukan swap
    kromosom[posisi1], kromosom[posisi2] = kromosom[posisi2], kromosom[posisi1]

    return kromosom
```

```

# Inversion Mutation
def inversion_mutation(kromosom):
    posisi1 = random.randint(0, len(kromosom) - 2)
    posisi2 = random.randint(posisi1 + 1, len(kromosom) - 1)
    # Perbaikan di sini: konversi hasil reversed ke list
    kromosom[posisi1:posisi2] =
list(reversed(kromosom[posisi1:posisi2]))
    return kromosom

# Uniform Mutation
def uniform_mutation(kromosom, mutation_rate=0.1):
    # Pastikan kromosom adalah list
    kromosom = list(kromosom) # Konversi ke list jika perlu

    for i in range(len(kromosom)):
        if random.random() < mutation_rate:
            kromosom[i] = 1 - kromosom[i] # Membalik nilai
gen
    return kromosom

# Definisikan anak1 sebelum digunakan
anak1 = [0, 1, 1, 0, 1] # Contoh kromosom, sesuaikan dengan
kebutuhan Anda

# Contoh penggunaan
mutasi_anak1 = swap_mutation(anak1.copy()) # Swap Mutation
mutasi_anak2 = inversion_mutation(anak1.copy()) # Inversion
Mutation
mutasi_anak3 = uniform_mutation(anak1.copy()) # Uniform
Mutation

# Menampilkan hasil setelah mutasi
print("\nAnak Setelah Mutasi:")
print(f"Anak 1 (Swap Mutation): {mutasi_anak1}")
print(f"Anak 2 (Inversion Mutation): {mutasi_anak2}")
print(f"Anak 3 (Uniform Mutation): {mutasi_anak3}")

```

Penjelasan Kode Python secara Detail:

- **Swap Mutation:**
 - **Parameter:**
 - **kromosom:** Kromosom yang akan dimutasi.
 - **Proses:**
 - Memilih dua posisi gen **posisi1** dan **posisi2** secara acak.

- Menukar gen pada posisi tersebut.
- **Return:**
 - Kromosom yang telah dimutasi.
- **Inversion Mutation:**
 - Memilih dua titik **posisi1** dan **posisi2** untuk menentukan segmen.
 - Membalik urutan gen dalam segmen tersebut.
- **Uniform Mutation:**
 - Untuk setiap gen, dengan probabilitas **mutation_rate**, membalik nilai gen tersebut (dari 0 menjadi 1 atau sebaliknya).
- **Contoh Penggunaan:**
 - Melakukan mutasi pada **anak1** menggunakan **swap_mutation**.
 - Menampilkan kromosom anak setelah mutasi.

Hasil dari kode tersebut akan menyerupai sebagai berikut dan bisa juga acak.

```
Anak Setelah Mutasi:
Anak 1 (Swap Mutation): [1, 1, 1, 0, 0]
Anak 2 (Inversion Mutation): [0, 1, 1, 0, 1]
Anak 3 (Uniform Mutation): [0, 1, 1, 0, 1]
```

6. Proses Pengulangan Generasi

Proses pengulangan generasi melibatkan iterasi proses evaluasi, seleksi, crossover, dan mutasi untuk sejumlah generasi tertentu. Tujuan utamanya adalah untuk menemukan solusi yang optimal atau mendekati optimal seiring dengan berjalannya generasi.

Setiap generasi terdiri dari langkah-langkah berikut:

- 1) **Evaluasi Fitness:** Menghitung nilai fitness setiap individu dalam populasi.
- 2) **Seleksi:** Memilih individu-individu yang akan menjadi parent.
- 3) **Crossover:** Menghasilkan anak-anak baru dari parent yang terpilih.

- 4) **Mutasi:** Melakukan mutasi pada anak-anak untuk mempertahankan keragaman genetik.
- 5) **Pembentukan Populasi Baru:** Menggantikan populasi lama dengan populasi baru yang terdiri dari anak-anak.

Pada Sebuah Folder dengan nama Algoritma Genetika , tambahkan file dengan Nama File `main.py`, kemudian ketikkan kode berikut. untuk import bisa

```
import random
import matplotlib.pyplot as plt
import numpy as np

# Mengimpor fungsi-fungsi dari file lain
from inisiasipopulasi import inisialisasi_populasi
from EvaluasiFitness import hitung_fitness
from selection import roulette_wheel_selection, tournament_selection
from crossover import one_point_crossover, two_point_crossover, uniform_crossover
from mutation import swap_mutation, inversion_mutation, uniform_mutation

# Data barang: (nama, nilai, berat)
barang = [
    ("Barang1", 60, 10),
    ("Barang2", 100, 20),
    ("Barang3", 120, 30),
    ("Barang4", 90, 25),
    ("Barang5", 69, 11),
    ("Barang6", 70, 9),
    ("Barang7", 80, 15),
    ("Barang8", 90, 10),
    ("Barang9", 25, 3)
]

def run_ga(jumlah_generasi, jumlah_populasi, prob_crossover, prob_mutasi,
kapasitas_tas):
    # Menentukan jumlah gen berdasarkan jumlah barang
    jumlah_gen = len(barang)

    # Inisialisasi populasi awal
    populasi = inisialisasi_populasi(jumlah_populasi, jumlah_gen)
    # Menghitung fitness untuk setiap individu dalam populasi
    fitness_populasi = [hitung_fitness(individu, barang, kapasitas_tas) for
individu in populasi]
    # List untuk menyimpan nilai fitness terbaik, terburuk, dan rata-rata setiap
generasi
    best_fitness_list = []
    worst_fitness_list = []
    avg_fitness_list = []
    all_fitness = []

    # Variabel untuk menyimpan individu terbaik secara keseluruhan
    best_individu = None
    best_fitness_overall = 0

    # Proses evolusi selama jumlah generasi yang ditentukan
    for generasi in range(jumlah_generasi):
        # Evaluasi fitness populasi saat ini
        fitness_populasi = [hitung_fitness(individu, barang, kapasitas_tas) for
individu in populasi]
```

```

# Menyimpan nilai fitness untuk plotting
best_fitness = max(fitness_populasi)
worst_fitness = min(fitness_populasi)
avg_fitness = sum(fitness_populasi) / len(fitness_populasi)
best_fitness_list.append(best_fitness)
worst_fitness_list.append(worst_fitness)
avg_fitness_list.append(avg_fitness)
all_fitness.append(fitness_populasi.copy())

# Menyimpan individu terbaik secara keseluruhan
if best_fitness > best_fitness_overall:
    best_fitness_overall = best_fitness
    index_best = fitness_populasi.index(best_fitness)
    best_individu = populasi[index_best]

new_populasi = []
used_indices = []

# Membentuk populasi baru
while len(new_populasi) < jumlah_populasi:
    # Seleksi orang tua menggunakan roulette wheel
    parent1, idx1 = roulette_wheel_selection(populasi, fitness_populasi)
    used_indices.append(idx1)
    # Memastikan orang tua kedua berbeda
    available_indices = [i for i in range(len(populasi)) if i not in
used_indices]
    if not available_indices:
        used_indices = [idx1]
        available_indices = [i for i in range(len(populasi)) if i !=
idx1]
    parent2, _ = roulette_wheel_selection([populasi[i] for i in
available_indices],[fitness_populasi[i] for i in available_indices])
    used_indices.append(available_indices[_])

    # Crossover untuk menghasilkan anak
    if random.random() < probab_crossover:
        anak1, anak2 = one_point_crossover(parent1, parent2)
    else:
        anak1, anak2 = parent1[:], parent2[:]

    # Mutasi pada anak
    if random.random() < probab_mutasi:
        anak1 = swap_mutation(anak1)
    if random.random() < probab_mutasi:
        anak2 = swap_mutation(anak2)

    # Menambahkan anak ke populasi baru
    new_populasi.extend([anak1, anak2])

# Memastikan populasi baru sesuai dengan jumlah populasi
populasi = new_populasi[:jumlah_populasi]

# Menampilkan grafik fitness
plt.figure(figsize=(12, 7))

# Plot semua nilai fitness dengan transparansi rendah
for i in range(jumlah_generasi):
    x = [i+1]*len(all_fitness[i])
    y = all_fitness[i]
    plt.scatter(x, y, color='gray', alpha=0.1)

# Plot nilai fitness terbaik, terburuk, dan rata-rata
plt.plot(range(1, jumlah_generasi+1), best_fitness_list, color='blue',
label='Fitness Tertinggi')

```

```

plt.plot(range(1, jumlah_generasi+1), worst_fitness_list, color='yellow',
label='Fitness Terendah')
plt.plot(range(1, jumlah_generasi+1), avg_fitness_list, color='red',
label='Fitness Rata-rata')

plt.title('Perkembangan Nilai Fitness')
plt.xlabel('Generasi')
plt.ylabel('Nilai Fitness')
plt.legend()
plt.grid(True)
plt.show()

# Menampilkan barang yang terpilih dalam knapsack terbaik
selected_items = [barang[i][0] for i in range(len(best_individu)) if
best_individu[i] == 1]
selected_value = hitung_fitness(best_individu, barang, kapasitas_tas)
selected_weight = sum([barang[i][2] for i in range(len(best_individu)) if
best_individu[i] == 1])

print(f"Nilai Fitness Terbaik: {selected_value}")
print(f"Total Bobot: {selected_weight}")
print("Barang Terpilih:")
for item in selected_items:
    print(f"- {item}")

# Menjalankan GA dengan parameter berikut
run_ga(
    jumlah_generasi=50,
    jumlah_populasi=20,
    prob_crossover=0.5,
    prob_mutasi=0.1,
    kapasitas_tas=50
)

```

Penjelasan Kode Python secara Detail:

Kode di atas merupakan implementasi dari Algoritma Genetika (Genetic Algorithm/GA) untuk menyelesaikan masalah Knapsack Problem, di mana tujuan utamanya adalah memilih kombinasi barang yang memiliki nilai (fitness) terbaik dengan tetap mempertimbangkan batasan kapasitas tas. Algoritma ini terdiri dari beberapa langkah utama, yaitu inisiasi populasi, seleksi, crossover, mutasi, dan evaluasi fitness pada setiap generasi.

Berikut adalah penjelasan rinci dari proses berjalannya program dari awal hingga akhir, serta bagaimana kromosom (solusi) bertransformasi dari inisiasi hingga menghasilkan solusi terbaik.

1) Inisialisasi

a. Barang dan Parameter

Sebelum algoritma berjalan, kita mendefinisikan daftar barang yang tersedia untuk dipilih. Setiap barang memiliki tiga atribut:

- **Nama barang** (misalnya, "Barang1"),
- **Nilai barang** atau keuntungan (misalnya, 60),
- **Berat barang** (misalnya, 10).

Parameter penting yang digunakan dalam algoritma adalah:

- **jumlah_generasi**: Jumlah iterasi (generasi) di mana populasi akan berevolusi.
- **jumlah_populasi**: Jumlah individu (solusi potensial) dalam setiap generasi.
- **prob_crossover**: Probabilitas terjadinya crossover (kombinasi dua individu untuk menghasilkan individu baru).
- **prob_mutasi**: Probabilitas mutasi (perubahan acak dalam kromosom).
- **kapasitas_tas**: Batasan berat maksimum yang dapat dibawa oleh tas (knapsack).

b. Inisialisasi Populasi

Di awal, algoritma menginisialisasi **populasi awal** secara acak. Setiap individu dalam populasi adalah kromosom yang merepresentasikan solusi potensial, di mana setiap **gen** dalam kromosom berisi 0 atau 1. Gen bernilai **1** berarti barang dipilih untuk dimasukkan ke dalam tas, sedangkan gen bernilai **0** berarti barang tidak dipilih.

```
populasi = inisialisasi_populasi(jumlah_populasi,  
jumlah_gen)
```

Pada tahap ini, sejumlah individu (kromosom) dihasilkan secara acak.

2) Evaluasi Fitness

Setiap individu dalam populasi dievaluasi untuk mengetahui seberapa baik solusi yang dihasilkan. Fungsi `hitung_fitness` menghitung nilai total barang yang dipilih dan membandingkannya dengan kapasitas tas. Jika total berat barang yang dipilih melebihi kapasitas tas, maka fitness individu tersebut dianggap nol (penalti).

```
fitness_populasi = [hitung_fitness(individu, barang,
kapasitas_tas) for individu in populasi]
```

3) Seleksi Orang Tua

Proses seleksi bertujuan untuk memilih dua individu dari populasi sebagai orang tua (parent) yang akan dikombinasikan menggunakan crossover untuk menghasilkan anak baru. Pada kode ini, digunakan Roulette Wheel Selection untuk memilih individu berdasarkan probabilitas proporsional terhadap nilai fitness.

Roulette Wheel Selection bekerja seperti roda putar, di mana setiap individu memiliki probabilitas untuk dipilih, yang sebanding dengan nilai fitness-nya. Individu dengan fitness lebih tinggi memiliki kemungkinan lebih besar untuk dipilih.

```
parent1, idx1 = roulette_wheel_selection(populasi,
fitness_populasi)
parent2, _ = roulette_wheel_selection([populasi[i] for i in
available_indices],[fitness_populasi[i] for i in
available_indices])
```

4) Crossover (Penyilangan)

Setelah dua orang tua dipilih, mereka dikombinasikan melalui proses Crossover untuk menghasilkan dua individu baru (anak). Pada kode ini digunakan One-Point Crossover, di mana titik potong acak ditentukan, dan bagian depan kromosom dari satu orang tua dikombinasikan dengan bagian belakang kromosom dari orang tua lainnya.

Jika probabilitas crossover tercapai (misalnya 0.5), proses ini dijalankan. Jika tidak, anak-anak adalah salinan dari orang tua tanpa perubahan.

```
if random.random() < probab_crossover:
    anak1, anak2 = one_point_crossover(parent1, parent2)
else:
    anak1, anak2 = parent1[:], parent2[:]
```

5) Mutasi

Setelah crossover, individu baru (anak) dapat mengalami mutasi dengan probabilitas tertentu (misalnya 10%). Mutasi adalah perubahan acak pada gen dalam kromosom, misalnya dari 1 menjadi 0, atau sebaliknya.

Mutasi membantu menjaga keragaman genetik dalam populasi, sehingga mencegah algoritma terjebak pada solusi lokal yang tidak optimal. Pada kode ini digunakan Swap Mutation untuk menukar dua gen dalam kromosom.

```
if random.random() < probab_mutas:
    anak1 = swap_mutation(anak1)
if random.random() < probab_mutas:
    anak2 = swap_mutation(anak2)
```

6) Pembentukan Populasi Baru

Setelah crossover dan mutasi, individu baru (anak-anak) ditambahkan ke dalam populasi baru. Proses ini diulang hingga populasi baru memiliki ukuran yang sama dengan populasi asli. Populasi baru menggantikan populasi lama pada generasi berikutnya.

```
new_populasi.extend([anak1, anak2])
populasi = new_populasi[:jumlah_populasi]
```

7) Evaluasi dan Hasil Akhir

Setiap generasi berakhir dengan evaluasi fitness dan penyimpanan individu terbaik. Proses ini diulang untuk jumlah generasi yang telah ditentukan. Pada akhir evolusi, individu terbaik ditampilkan, beserta nilai fitness-nya dan barang-barang yang terpilih dalam knapsack.

```
selected_items = [barang[i][0] for i in
range(len(best_individu)) if best_individu[i] == 1]
selected_value = hitung_fitness(best_individu, barang,
kapasitas_tas)
selected_weight = sum([barang[i][2] for i in
range(len(best_individu)) if best_individu[i] == 1])

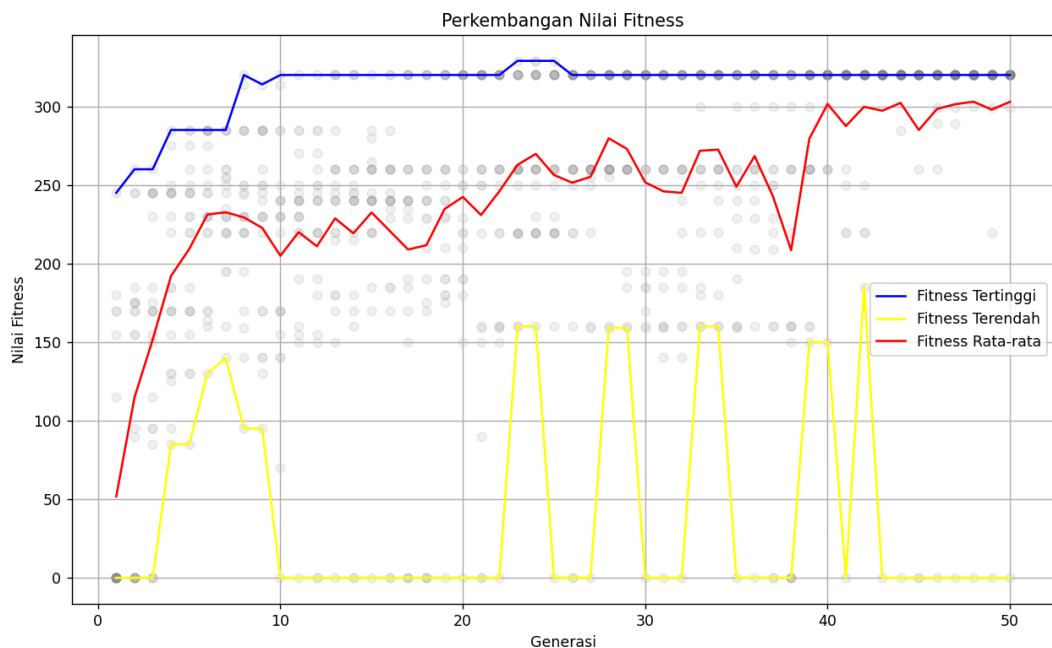
print(f"Nilai Fitness Terbaik: {selected_value}")
```

```
print(f"Total Bobot: {selected_weight}")
print("Barang Terpilih:")
for item in selected_items:
    print(f"- {item}")
```

8) Hasil Grafik

Hasil Grafik ditunjukkan dengan 3 warna yaitu warna kuning (titik fitness terendah), warna merah (rerata titik fitness), warna biru (titik fitness tertinggi). Serta titik abu-abu menunjukan nilai fitness individu yang lainnya, dimana semakin gelap warna abu-abunya berarti semakin banyak individu dengan fitness yang serupa.

Note : Nilai yang acak dan tidak rata menunjukan bahwa GA itu memiliki sifat variatif yang tinggi dan memungkinkan segala kemungkinan lain baik naik atau turun.



9) Hasil Knapsack Terpilih dengan Fitness Terbaik

Pada Terminal Bagian paling bawah ditunjukkan Barang Barang Terpilih oleh Knapsack menggunakan GA. GA memilih barang dengan nilai fitness terbaik namun maxsizenya mencukupi batas tertinggi.

Hasil sebagai berikut :

```
Nilai Fitness Terbaik: 329
Total Bobot: 50
Barang Terpilih:
- Barang2
- Barang5
- Barang6
- Barang8
```

TUGAS

Upload source code percobaan di atas ke GitHub dengan nama repositori sebagai berikut,

NIM-PraktikumKB-Pertemuan9

(Tugas diselesaikan di waktu praktikum sekarang juga)

Terimakasih, Sampai Jumpa 🍷